



Research integration patterns for plugin systems for modern Next.js App Router applications

Modern Next.js App Router applications can integrate plugin systems using several effective patterns focusing on modularity, dynamic loading, and compatibility with the new React Server Components architecture.

Dynamic Plugin Loading

A common pattern is to dynamically import plugin components based on route parameters. This allows loading plugins on-demand, improving performance and maintainability. For example, a route can dynamically import a plugin component using Next.js dynamic imports with fallback loading UI, enabling a plugin architecture where new plugins can be added without changing core application code. This approach works well in the App Router context where routes can be nested and modularized for each plugin segment.

Colocation and Modular Architecture

Next.js App Router encourages colocating logic and UI with routes using the file system (placing `loading.js`, `error.js`, `route.js` files alongside route components). Plugins can be structured as separate route groups or features within the `app` directory, each with their own layouts, pages, and API routes. This keeps plugins modular and maintainable. Using a shared `config` or `lib` directory for common plugin utilities or interfaces supports consistency.

Handling Client and Server Components

Since the App Router uses React Server Components by default, plugins must carefully distinguish between server and client components. Components that require interactivity must be prefixed with `"use client";` and can be nested inside primarily server-rendered plugin code. This enables performant server-side rendering for most plugin logic, falling back to client rendering where necessary.

Plugin System Integration Example

One real-world approach uses a dedicated route pattern like `/plugins/[plugin]/[page]` that dynamically imports plugin modules based on URL. Plugin code is often located outside the main `app` directory and symlinked into the `app` during build. Webpack or Next.js config ensures the plugin paths are compiled and resolved correctly during development and production builds.

Compatibility Considerations

With the App Router and React Server Components, plugin authors must ensure compatibility by avoiding client-only libraries in server components and testing plugin usage thoroughly in the App Router context. Some older UI or state management libraries might need adaptation or running in client components only.

In summary, modern Next.js App Router plugin integration patterns focus on dynamic imports, modular route-based structure, server/client component boundaries, and build-time tooling for plugin discovery and compilation. This enables scalable and maintainable plugin systems optimized for Next.js's latest architecture.

References to these patterns and example code snippets can be found in recent developer guides, blog posts, and Next.js documentation on App Router architecture and plugin handling approaches. [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)



Explore best practices for modular plugin architecture in Next.js

Best practices for modular plugin architecture in Next.js focus on creating isolated, reusable, and scalable feature modules that can be independently developed, tested, and deployed. Key principles include:

Modular Component Design

Plugins should consist of independent, reusable components that avoid tightly coupled dependencies or global state. This promotes easy maintenance and reuse across projects. Components are organized into self-contained feature folders with a clear boundary between UI, business logic, and data access.

Feature Isolation and Encapsulation

Each plugin or feature module should be isolated to prevent interference with other modules. This can be done by:

- Encapsulating data fetching and state management locally within the plugin.
- Using separate route groups or namespaces for plugins to avoid routing conflicts.
- Leveraging Next.js server components alongside client components carefully for performance and interactivity separation.

Lazy Loading and Code Splitting

To improve loading performance, plugins should be lazily loaded using Next.js dynamic imports. This ensures plugin code is fetched only when the feature is accessed, reducing initial bundle size and improving user experience.

Type Safety and API Layer

A type-safe API layer using TypeScript and runtime validation libraries (e.g., Zod) helps ensure data integrity between plugins and backend services. Separating client and server API logic also enhances maintainability and performance.

Independent Versioning and Deployment

Treat plugins as independently versioned modules, possibly maintained as git submodules or npm packages. This enables teams to iterate and deploy features without impacting the entire application.

Error Boundaries and Testing

Implement error boundaries within plugins to isolate failures and prevent app-wide crashes. Isolate plugin tests to validate UI, business logic, and API integration in a modular manner.

Project Structure Example

```
app/  
  (protected)/  
    products/  
      page.tsx          # Listing plugin UI  
      create/page.tsx   # Feature-specific UI  
      _components/      # Reusable UI pieces  
    settings/  
      notifications/    # Another plugin  
lib/  
  api/                  # Shared API utilities  
  types/                # Shared types and validation
```

These practices enable scalable development where features/plugins can be composed, added, or removed with minimal impact on other parts of the app. This is highly aligned with Next.js 15+ modern architecture, leveraging App Router, server components, and code splitting for optimal performance.

This guidance aligns with recent expert discussions and Next.js best practices on modular architecture, lazy loading, and isolated plugin development. [\[21\]](#) [\[22\]](#) [\[23\]](#) [\[24\]](#)

How to implement feature isolation in Next.js plugins

Implementing feature isolation in Next.js plugins involves several key strategies to ensure that each plugin or feature module operates independently without impacting other parts of the application.

Separate Routing and Namespaces

Use Next.js App Router's nested routing feature to isolate plugins into their own route groups or namespaces, for example, `/plugins/[pluginName]/` or `/features/[featureName]/`. This helps avoid route conflicts and keeps the URL structure clear and organized.

Encapsulated State and Data Management

Keep state and data fetching logic localized within the plugin component or module. Use React context or local state management inside the plugin rather than global stores to avoid cross-feature state leakage. For server data, use Next.js's `fetch` or `use` within the plugin scope.

Component and UI Encapsulation

Organize plugin UI components in separate folders and avoid sharing mutable UI components or styles globally. Prefer scoped CSS modules or styled components within the plugin directory to prevent style leakage.

Isolated API Routes and Services

Define plugin-specific API routes within their own API route segments under the plugin namespace, e.g., `app/api/plugins/[pluginName]/route.js`. This isolates backend logic and avoids service conflicts.

Error Boundaries and Fallbacks

Wrap plugin components with React error boundaries to catch errors within the plugin and show isolated fallback UI. This prevents one plugin's failure from crashing the entire app.

Use React Server Components Boundaries

Leverage React Server Components in Next.js so that the majority of plugin logic runs on the server, with client components used strictly where needed (e.g., for interactivity). This naturally isolates execution environments and reduces unintended side effects.

Example Folder Structure

```
app/  
  plugins/  
    chat/  
      page.tsx
```

```
_components/  
api/  
  route.ts  
analytics/  
  page.tsx  
_components/
```

By following these practices, each Next.js plugin can function independently with clear boundaries around routing, state, UI, APIs, and error handling, enabling safer modular architecture and easier maintenance.

These approaches align with modern Next.js App Router architecture and best practice recommendations for scalable modular development. [\[41\]](#) [\[42\]](#) [\[43\]](#) [\[44\]](#)

✱✱

Runtime Plugin System Patterns for Next.js

Runtime plugin systems in Next.js present unique challenges due to the framework's static compilation and routing model. However, several patterns have emerged for implementing dynamic, extensible plugin architectures.

Module Federation

Module Federation, enabled by Webpack 5's `ModuleFederationPlugin`, is the most mature solution for runtime plugin loading in Next.js. This approach allows separate Next.js applications to share code dynamically at runtime without requiring a rebuild. [\[45\]](#) [\[46\]](#)

Implementation with `@module-federation/nextjs-mf`:

The `NextFederationPlugin` extends Next.js to support federated modules for both client and server-side rendering. Configure a host application to consume remote plugins:

```
// next.config.js (Host)  
const NextFederationPlugin = require('@module-federation/nextjs-mf');  
  
module.exports = {  
  webpack(config, options) {  
    config.plugins.push(  
      new NextFederationPlugin({  
        name: 'host',  
        remotes: {  
          pluginA: `pluginA@http://localhost:3001/_next/static/${  
            options.isServer ? 'ssr' : 'chunks'  
          }}/remoteEntry.js`,  
        },  
        shared: {},  
      })  
    );  
    return config;  
  }  
};
```

```
}  
};
```

Remote plugins expose components that can be imported dynamically:

```
// Consuming a federated plugin  
import dynamic from 'next/dynamic';  
  
const RemotePlugin = dynamic(() => import('pluginA/Widget'), {  
  ssr: false  
});
```

Limitations: Module Federation works best for distributing plugins as separate deployments. As of Next.js 15, some developers report compatibility issues that require careful version management.^[47]

Dynamic Component Registry Pattern

For plugins within a single application, a registry pattern combined with dynamic imports provides runtime extensibility. This approach uses a centralized plugin registry where plugins self-register their components and metadata.^[48] ^[49]

Core Architecture:

1. **Plugin Contract** - Define a standard interface all plugins must implement:

```
// Plugin interface  
export default {  
  id: 'my-plugin',  
  name: 'My Plugin',  
  version: '1.0.0',  
  component: () => import('./PluginComponent'),  
  metadata: {  
    description: 'Plugin description',  
    dependencies: []  
  }  
};
```

2. **Plugin Registry with Context** - Manage available plugins and provide them via React Context:

```
// lib/plugins/PluginRegistry.js  
class PluginRegistry {  
  constructor() {  
    this.plugins = new Map();  
  }  
  
  register(plugin) {  
    if (this.plugins.has(plugin.id)) {  
      throw new Error(`Plugin ${plugin.id} already registered`);  
    }  
  }  
}
```

```

    this.plugins.set(plugin.id, plugin);
  }

  async load(pluginId) {
    const plugin = this.plugins.get(pluginId);
    if (!plugin) return null;

    const { default: Component } = await plugin.component();
    return { ...plugin, Component };
  }

  getAll() {
    return Array.from(this.plugins.values());
  }
}

export const registry = new PluginRegistry();

```

3. Plugin Provider - Load plugins at runtime and expose them to the app:

```

'use client';

import { createContext, useContext, useEffect, useState } from 'react';
import { registry } from '@lib/plugins/PluginRegistry';

const PluginContext = createContext([]);

export function PluginProvider({ children, pluginPaths }) {
  const [plugins, setPlugins] = useState([]);

  useEffect(() => {
    async function loadPlugins() {
      const loaded = await Promise.all(
        pluginPaths.map(async (path) => {
          const mod = await import(`@/plugins/${path}`);
          registry.register(mod.default);
          return registry.load(mod.default.id);
        })
      );
      setPlugins(loaded.filter(Boolean));
    }
    loadPlugins();
  }, [pluginPaths]);

  return (
    <PluginContext.Provider value={plugins}>
      {children}
    </PluginContext.Provider>
  );
}

export const usePlugins = () => useContext(PluginContext);

```

4. Dynamic Rendering - Render plugins based on route or user configuration:

```

'use client';

import { usePlugins } from '@lib/plugins/PluginProvider';

export function PluginHost() {
  const plugins = usePlugins();

  return (
    <div className="plugin-container">
      {plugins.map(({ id, Component, name }) => (
        <div key={id} className="plugin-wrapper">
          <h3>{name}</h3>
          <Component />
        </div>
      ))}
    </div>
  );
}

```

Event-Driven Plugin Communication

For plugins that need to communicate, implement an event bus using Node.js EventEmitter or a custom pub/sub pattern. This decouples plugins from each other while enabling coordination. [\[50\]](#) [\[51\]](#) [\[52\]](#)

```

// lib/plugins/EventBus.js
import { EventEmitter } from 'events';

class PluginEventBus extends EventEmitter {
  constructor() {
    super();
    this.setMaxListeners(50); // Increase for many plugins
  }

  registerPlugin(pluginId, handlers) {
    Object.entries(handlers).forEach(([event, handler]) => {
      this.on(event, (data) => {
        handler({ ...data, sourcePlugin: pluginId });
      });
    });
  }
}

export const eventBus = new PluginEventBus();

```

Plugins subscribe to events during initialization:

```

// plugins/notificationPlugin.js
export default {
  id: 'notifications',
  init(context) {
    context.eventBus.registerPlugin('notifications', {

```



```

    'user:created': (data) => {
      console.log('New user created:', data);
      // Send notification
    }
  });
}
};

```

Database-Driven Plugin Configuration

For multi-tenant SaaS applications, store plugin configurations in a database and load them at request time. This enables per-tenant or per-user plugin customization without redeployment. [\[53\]](#)

```

// app/api/plugins/route.js
import { getPluginsForTenant } from '@lib/db';

export async function GET(request) {
  const tenantId = request.headers.get('x-tenant-id');
  const plugins = await getPluginsForTenant(tenantId);

  return Response.json({
    plugins: plugins.map(p => ({
      id: p.id,
      enabled: p.enabled,
      config: p.config
    }))
  });
}

```

Client-side dynamic loading based on API response:

```

'use client';

import { useEffect, useState } from 'react';

export function TenantPluginHost() {
  const [plugins, setPlugins] = useState([]);

  useEffect(() => {
    fetch('/api/plugins')
      .then(r => r.json())
      .then(data => {
        Promise.all(
          data.plugins
            .filter(p => p.enabled)
            .map(async (p) => {
              const mod = await import(`@/plugins/${p.id}`);
              return { ...mod.default, config: p.config };
            })
        ).then(setPlugins);
      });
  }, []);
}

```

```

    return (
      <div>
        {plugins.map(({ id, Component, config }) => (
          <Component key={id} config={config} />
        ))}
      </div>
    );
  }
}

```

Dependency Injection for Plugin Services

Implement dependency injection to provide shared services (API clients, authentication, storage) to plugins without tight coupling. This keeps plugins portable and testable. [\[54\]](#) [\[55\]](#) [\[56\]](#)

```

// lib/plugins/ServiceContainer.js
class ServiceContainer {
  constructor() {
    this.services = new Map();
  }

  register(name, factory) {
    this.services.set(name, { factory, instance: null });
  }

  get(name) {
    const service = this.services.get(name);
    if (!service) throw new Error(`Service ${name} not found`);

    if (!service.instance) {
      service.instance = service.factory(this);
    }
    return service.instance;
  }
}

export const container = new ServiceContainer();

// Register core services
container.register('api', () => new ApiClient());
container.register('auth', (c) => new AuthService(c.get('api')));

```

Plugins receive services via initialization:

```

// plugins/analyticsPlugin.js
export default {
  id: 'analytics',
  init(services) {
    const api = services.get('api');
    const auth = services.get('auth');

    return {
      trackEvent: (event) => {

```

```

    api.post('/analytics', {
      event,
      userId: auth.getCurrentUser().id
    });
  }
};
}
};

```

Runtime Limitations and Workarounds

Challenge: Next.js routes are determined at build time, making true runtime route registration impossible. [\[57\]](#) [\[53\]](#)

Workaround: Use catch-all routes with dynamic plugin resolution:

```

// app/plugins/[...slug]/page.js
export default async function PluginPage({ params }) {
  const { slug } = await params;
  const pluginId = slug[0];

  const plugin = await getPluginById(pluginId);
  if (!plugin) notFound();

  const { default: Component } = await import(`@/plugins/${pluginId}`);
  return <Component route={slug.slice(1)} />;
}

```

Challenge: Server-side plugin execution requires careful handling of Node.js vs. Edge Runtime. [\[58\]](#) [\[59\]](#) [\[57\]](#)

Workaround: Mark plugin routes explicitly with runtime exports and validate plugin compatibility:

```

// Validate plugin runtime compatibility
export const validatePlugin = (plugin) => {
  if (plugin.runtime === 'edge' && plugin.usesNodeAPIs) {
    throw new Error(`Plugin ${plugin.id} requires Node.js runtime`);
  }
};

```

Best Practices Summary

Lazy Loading: Always use Next.js `dynamic()` imports for plugin components to enable code splitting and on-demand loading. [\[60\]](#) [\[61\]](#)

Validation: Validate plugin structure and dependencies before loading to prevent runtime errors. [\[51\]](#) [\[50\]](#)

Isolation: Use error boundaries around plugin components to contain failures. [\[62\]](#) [\[63\]](#)

Versioning: Track plugin versions and API compatibility to manage breaking changes across plugin updates. [\[50\]](#)

Documentation: Provide clear plugin APIs and hooks so third-party developers can build compatible extensions. [\[49\]](#) [\[51\]](#)

Runtime plugin systems require balancing flexibility with Next.js's compilation model. Module Federation offers the most powerful runtime capabilities but with deployment complexity, while registry-based patterns provide simpler integration for internally managed plugins. Choose based on whether you need true multi-deployment plugin distribution or single-app extensibility. [\[46\]](#) [\[53\]](#) [\[45\]](#) [\[48\]](#) [\[51\]](#)



1. <https://leapcell.io/blog/navigating-next-js-app-router-and-pages-router-evolution>
2. <https://www.grouparoo.com/blog/nextjs-plugins>
3. <https://makerkit.dev/blog/tutorials/nextjs-app-router-project-structure>
4. <https://nextjs.org/docs/app>
5. <https://nextjs.org/docs/14/app/building-your-application/data-fetching/patterns>
6. <https://dev.to/yukionishi1129/building-a-production-ready-nextjs-app-router-architecture-a-complete-playbook-3f3h>
7. <https://nextjs.org/docs/architecture>
8. <https://nx.dev/docs/technologies/react/next/guides/next-config-setup>
9. <https://javascript.plainenglish.io/next-js-software-design-architecture-best-practices-855fc4ec806d>
10. <https://next-intl.dev/docs/getting-started/app-router>
11. <https://nextjs.org/docs/architecture/nextjs-compiler>
12. <https://www.bigscal.com/blogs/frontend/top-must-nextjs-plugins-enhance-web-development/>
13. <https://www.raftlabs.com/blog/building-with-next-js-best-practices-and-benefits-for-performance-first-teams/>
14. <https://www.youtube.com/watch?v=EbrBRQ6UblY>
15. https://www.reddit.com/r/nextjs/comments/1n88lnr/does_anyone_actually_use_the_nextjs_app_router_as/
16. https://dev.to/nithya_ayer/5-design-patterns-for-building-scalable-nextjs-applications-1c80
17. <https://www.patterns.dev/react/nextjs/>
18. <https://stackademic.com/blog/5-design-patterns-for-building-scalable-next-js-applications>
19. <https://strapi.io/blog/nextjs-libraries>
20. <https://upsun.com/blog/avoid-common-mistakes-with-next-js-app-router/>
21. <https://rakesh.tembhurne.com/blog/coding/building-plugin-architecture-nextjs-15>
22. https://dev.to/nithya_ayer/5-design-patterns-for-building-scalable-nextjs-applications-1c80
23. <https://blog.logrocket.com/structure-scalable-next-js-project-architecture/>
24. <https://css-tricks.com/designing-a-javascript-plugin-system/>
25. <https://github.com/vercel/next.js/discussions/77862>

26. <https://stackoverflow.com/questions/10763006/plugin-architecture-in-web-apps-examples-or-code-snippets>
27. <https://stackoverflow.com/questions/76214501/nextjs-13-folder-structure-best-practice>
28. <https://news.ycombinator.com/item?id=20770105>
29. https://www.reddit.com/r/reactjs/comments/16iwd8p/how_can_i_make_my_nextjs_app_look_modern_and/
30. <https://nextjs.org/docs/architecture/nextjs-compiler>
31. <https://kanhasoft.com/blog/how-to-build-scalable-saas-products-with-next-js/>
32. <https://www.raftlabs.com/blog/building-with-next-js-best-practices-and-benefits-for-performance-first-teams/>
33. https://www.reddit.com/r/reactjs/comments/1blyfp3/plugin_system_design_pattern_like_obsidian/
34. <https://www.youtube.com/watch?v=lu5aZDqZt8E>
35. <https://nextjs.org>
36. <https://softwaremill.com/modern-full-stack-application-architecture-using-next-js-15/>
37. https://www.reddit.com/r/nextjs/comments/1g4n05z/best_practices_for_separation_of_concerns_mvcm_vvm/
38. <https://nextjs.org/docs/architecture>
39. <https://dev.to/alexeagleson/how-to-build-scalable-architecture-for-your-nextjs-project-2pb7>
40. <https://blog.pixelfreestudio.com/how-to-build-scalable-applications-with-next-js/>
41. <https://rakesh.tembhurne.com/blog/coding/building-plugin-architecture-nextjs-15>
42. <https://blog.logrocket.com/structure-scalable-next-js-project-architecture/>
43. https://dev.to/nithya_iyer/5-design-patterns-for-building-scalable-nextjs-applications-1c80
44. <https://nextjs.org/docs/app>
45. <https://module-federation.io/guide/framework/nextjs>
46. <https://module-federation.io/practice/frameworks/next/>
47. https://www.reddit.com/r/nextjs/comments/1ka2bjm/can_i_able_to_use_module_federation_somewhat_in/
48. <https://dev.to/hexshift/building-a-plugin-system-in-react-using-dynamic-imports-and-context-api-3j6e>
49. <https://css-tricks.com/designing-a-javascript-plugin-system/>
50. <https://github.com/supnate/js-plugin>
51. <https://www.n-school.com/plugin-based-architecture-in-node-js/>
52. <https://aws.amazon.com/event-driven-architecture/>
53. https://www.reddit.com/r/nextjs/comments/1fg34qr/runtime_pluginmodular_routes_in_nextjs/
54. <https://snky.io/blog/dependency-injection-in-javascript/>
55. <https://dev.to/walosha/dependency-injection-pattern-for-beginners-3nc4>
56. <https://blog.risingstack.com/dependency-injection-in-node-js/>
57. <https://stackoverflow.com/questions/72113316/nextjs-middlewares-use-default-runtime-instead-of-edge-runtime>
58. <https://nextjs.org/docs/app/api-reference/edge>
59. <https://nextjs.org/docs/13/app/building-your-application/rendering/edge-and-nodejs-runtimes>

60. <https://nextjs.org/docs/pages/guides/lazy-loading>
61. <https://www.oneclickitsolution.com/centerofexcellence/reactjs/nextjs-dynamic-loading-reduce-bundle-size-improve-performance>
62. <https://rakesh.tembhurne.com/blog/coding/building-plugin-architecture-nextjs-15>
63. <https://blog.logrocket.com/structure-scalable-next-js-project-architecture/>
64. <https://nextjs.org/docs/architecture/nextjs-compiler>
65. <https://github.com/module-federation/core/issues/1939>
66. <https://www.serverless.com/plugins/serverless-nextjs-plugin>
67. <https://strapi.io/blog/extensibility-in-software-engineering>
68. <https://opennext.js.org/aws/contribute/plugin>
69. <https://workflowengine.io/documentation/dynamic-plugin-loading>
70. <https://www.ssw.com.au/rules/rules-to-better-nextjs/>
71. <https://stackoverflow.com/questions/44778265/dynamically-loading-react-components>
72. <https://blog.bitsrc.io/frontend-architecture-a-complete-guide-to-building-scalable-next-js-applications-d28b0000e2ee>
73. <https://nextjs.org/blog/next-16>
74. <https://nextjs.org/learn/seo/dynamic-import-components>
75. <https://arno.surfacew.com/posts/nextjs-architecture>
76. <https://docs.netlify.com/build/frameworks/framework-setup-guides/nextjs/overview/>
77. <https://stackoverflow.com/questions/77388042/how-to-dynamically-configure-the-runtime-for-nextjs-api-routes>
78. <https://nextjs.org/docs/13/app/building-your-application/routing/dynamic-routes>
79. <https://nextjs.org/docs/app/api-reference/file-conventions/dynamic-routes>
80. <https://webpack.js.org/plugins/module-federation-plugin/>
81. <https://tsh.io/blog/next-js-dynamic-content/>
82. <https://github.com/vercel/next.js/discussions/41309>
83. https://stacks.ensono.com/docs/module_federation/nextjs_plugin
84. <https://www.youtube.com/watch?v=iy6AJzldUw>
85. <https://stackoverflow.com/questions/68704673/whats-the-most-practical-way-to-render-dynamic-components-in-next-js>
86. <https://nextjs.org/docs/pages/building-your-application/routing/dynamic-routes>
87. <https://nextjs.org/docs/pages/guides/babel>
88. <https://www.privjs.com/packages/@module-federation/nextjs-mf>
89. https://www.reddit.com/r/reactjs/comments/1ne6m6w/is_it_possible_to_render_react_components_from_a/
90. <https://github.com/vercel/next.js/discussions/33327>
91. <https://javascript.plainenglish.io/function-registry-pattern-explained-clean-scalable-composable-code-e483bb7f2444>
92. <https://stackoverflow.com/questions/16315127/design-pattern-for-self-registering-components-plugins>
93. https://dev.to/nithya_ier/5-design-patterns-for-building-scalable-nextjs-applications-1c80

94. <https://nextjs.org/docs/14/app/building-your-application/rendering/composition-patterns>
95. <https://makerkit.dev/blog/tutorials/nextjs-app-router-project-structure>
96. <https://www.geeksforgeeks.org/reactjs/build-an-online-marketplace-using-nextjs/>
97. <https://www.youtube.com/watch?v=j-wA6TKtT0Q>
98. <https://dev.to/melvinprince/the-complete-guide-to-scalable-nextjs-architecture-39o0>
99. https://www.reddit.com/r/nextjs/comments/1bo6eq4/how_to_build_a_plugin_marketplace_into_my_software/
100. <https://www.patterns.dev/react/nextjs/>
101. <https://javascript.plainenglish.io/design-extensible-react-application-architecture-1c491e9ed525>
102. <https://www.youtube.com/watch?v=6fXNWBFPfRM>
103. https://www.reddit.com/r/nextjs/comments/1gxeowu/design_patterns_in_nextjs_and_reactjs/
104. <https://makerkit.dev/docs/next-supabase/architecture/architecture>
105. <https://www.youtube.com/watch?v=dHuRDF6HD18>
106. <https://www.saffrontech.net/blog/10-best-nextjs-plugins-and-extensions>
107. <https://softwaremill.com/modern-full-stack-application-architecture-using-next-js-15/>
108. <https://www.youtube.com/watch?v=06g6YJ6JCJU>
109. https://www.reddit.com/r/learnjavascript/comments/sujp67/plugin_architecture/
110. <https://stackoverflow.com/questions/10763006/plugin-architecture-in-web-apps-examples-or-code-snippets>
111. <https://modernjs.dev/plugin/plugin-system>
112. <https://developer.adobe.com/app-builder/docs/resources/event-driven/>
113. <https://stackoverflow.com/questions/35983036/design-pattern-to-implement-plugins-that-depend-on-other-plugins>
114. <https://news.ycombinator.com/item?id=20770105>
115. <https://www.redhat.com/en/blog/event-driven-architecture-modern-applications>
116. <https://dev.to/arcanis/plugin-systems-when-why-58pp>
117. <https://www.tinybird.co/blog/event-driven-architecture-best-practices-for-databases-and-files>
118. <https://laurentcazanove.com/blog/vue-dependency-injection>
119. <https://www.confluent.io/learn/event-driven-architecture/>
120. https://www.reddit.com/r/softwarearchitecture/comments/pcugb7/whats_the_architecture_of_a_plugin_based_saas_web/
121. <https://blog.appsignal.com/2022/02/16/dependency-injection-in-javascript-write-testable-code-easily.html>